

▼ Train and Deploy a CNN Model Using TensorFlow Serving

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Set Google Drive paths

```
import os

project_path = '/content/drive/MyDrive/MSIL Project/ADL'

os.listdir(project_path)
```

```
['1753252736_train_and_deploy_a_cnn_model_using_tensorflow_serving_problem_statement.pdf',
'Datasets.zip',
'retinopathy_data',
'Train and Deploy a CNN Model Using TensorFlow Serving.ipynb']
```

Find the exact file location

```
import os

for root, dirs, files in os.walk("/content/drive/MyDrive"):
    if "Datasets.zip" in files or "Train and Deploy a CNN Model Using TensorFlow Serving.ipynb" in files:
        print("\nFOLDER:", root)

        if "Datasets.zip" in files:
            print("DATASET:", os.path.join(root, "Datasets.zip"))

        if "Train and Deploy a CNN Model Using TensorFlow Serving.ipynb" in files:
            print("NOTEBOOK:", os.path.join(
                root,
                "Train and Deploy a CNN Model Using TensorFlow Serving.ipynb"
            ))
```

```
FOLDER: /content/drive/MyDrive/MSIL Project/Capstone
DATASET: /content/drive/MyDrive/MSIL Project/Capstone/Datasets.zip
```

```
FOLDER: /content/drive/MyDrive/MSIL Project/NLP
DATASET: /content/drive/MyDrive/MSIL Project/NLP/Datasets.zip
```

```
FOLDER: /content/drive/MyDrive/MSIL Project/ADL
DATASET: /content/drive/MyDrive/MSIL Project/ADL/Datasets.zip
NOTEBOOK: /content/drive/MyDrive/MSIL Project/ADL/Train and Deploy a CNN Model Using TensorFlow Serving.ipynb
```

Define the ADL project paths

```
import os

PROJECT_PATH = "/content/drive/MyDrive/MSIL Project/ADL"

DATASET_ZIP = os.path.join(PROJECT_PATH, "Datasets.zip")
PROJECT_NOTEBOOK = os.path.join(
    PROJECT_PATH,
    "Train and Deploy a CNN Model Using TensorFlow Serving.ipynb"
)

print("Dataset ZIP exists:", os.path.exists(DATASET_ZIP))
print("Notebook exists:", os.path.exists(PROJECT_NOTEBOOK))
print("Dataset path:", DATASET_ZIP)
```

```
Dataset ZIP exists: True
Notebook exists: True
Dataset path: /content/drive/MyDrive/MSIL Project/ADL/Datasets.zip
```

Inspect the ZIP structure

```
import zipfile
import os
```

```

with zipfile.ZipFile(DATASET_ZIP, "r") as zip_ref:
    file_list = zip_ref.namelist()

print("Total files/folders in ZIP:", len(file_list))

print("\nFirst 30 items:")
for item in file_list[:30]:
    print(item)

```

Total files/folders in ZIP: 41

First 30 items:

```

Assisted_Practices_datasets/
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Detecting_Lines/
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Detecting_Lines/CurvedRoad.jpg
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Detecting_Lines/Highway.jpg
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Detecting_Lines/line and circle.png.png
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Detecting_Lines/NightRoad.jpg
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Drawing_Contour/
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Drawing_Contour/horse_new.png
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Drawing_Rectangles/
Assisted_Practices_datasets/Lesson_02_Image_Formation_and_Processing/Drawing_Rectangles/green_tree.jpg
Assisted_Practices_datasets/Lesson_05_Object_Detection/
Assisted_Practices_datasets/Lesson_05_Object_Detection/coco.names
Assisted_Practices_datasets/Lesson_05_Object_Detection/test.jpg
Assisted_Practices_datasets/Lesson_05_Object_Detection/yolov3.cfg
Assisted_Practices_datasets/Lesson_08_Generating_Images_with_Neural_Style/
Assisted_Practices_datasets/Lesson_08_Generating_Images_with_Neural_Style/content.jpg
Assisted_Practices_datasets/Lesson_08_Generating_Images_with_Neural_Style/style.jpg
Assisted_Practices_datasets/Lesson_10_OCR/
Assisted_Practices_datasets/Lesson_10_OCR/Detecting_Text_from_a_Handwritten_Notes/
Assisted_Practices_datasets/Lesson_10_OCR/Detecting_Text_from_a_Handwritten_Notes/AP_1.jpg
Assisted_Practices_datasets/Lesson_10_OCR/Prepare_the_Image_for_OCR/
Assisted_Practices_datasets/Lesson_10_OCR/Prepare_the_Image_for_OCR/AP_2.png
Assisted_Practices_datasets/Lesson_12_Explainable/
Assisted_Practices_datasets/Lesson_12_Explainable/image.png
Course_end_project_datasets/
Course_end_project_datasets/CEP_01/
Course_end_project_datasets/CEP_01/Sample.zip
Course_end_project_datasets/CEP_01/trainLabels.csv
Course_end_project_datasets/CEP_02/

```

Check the extracted files

```

import os
import zipfile

# Create a working folder in Google Drive
DATA_DIR = os.path.join(PROJECT_PATH, "retinopathy_data")
os.makedirs(DATA_DIR, exist_ok=True)

# Extract only CEP_01 files from the main ZIP
with zipfile.ZipFile(DATASET_ZIP, "r") as zip_ref:
    for file_name in zip_ref.namelist():
        if file_name.startswith("Course_end_project_datasets/CEP_01/"):
            zip_ref.extract(file_name, DATA_DIR)

print("Required project files extracted.")

```

Required project files extracted.

Check the extracted files

```

import os

EXTRACTED_CEP_PATH = os.path.join(
    DATA_DIR,
    "Course_end_project_datasets",
    "CEP_01"
)

print("CEP_01 folder:", EXTRACTED_CEP_PATH)

print("\nFiles inside CEP_01:")
for item in os.listdir(EXTRACTED_CEP_PATH):
    print("-", item)

SAMPLE_ZIP = os.path.join(EXTRACTED_CEP_PATH, "Sample.zip")
LABELS_CSV = os.path.join(EXTRACTED_CEP_PATH, "trainLabels.csv")

```

```
print("\nSample.zip exists:", os.path.exists(SAMPLE_ZIP))
print("trainLabels.csv exists:", os.path.exists(LABELS_CSV))
```

CEP_01 folder: /content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/Course_end_project_datasets/CEP_01

Files inside CEP_01:

- Sample.zip
- trainLabels.csv

Sample.zip exists: True

trainLabels.csv exists: True

Inspect the images inside Sample.zip

```
import zipfile

with zipfile.ZipFile(SAMPLE_ZIP, "r") as zip_ref:
    image_files = [
        f for f in zip_ref.namelist()
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ]

print("Total images inside Sample.zip:", len(image_files))

print("\nFirst 20 image files:")
for file in image_files[:20]:
    print(file)
```

Total images inside Sample.zip: 10

First 20 image files:

sample/10_left.jpeg
sample/10_right.jpeg
sample/13_left.jpeg
sample/13_right.jpeg
sample/15_left.jpeg
sample/15_right.jpeg
sample/16_left.jpeg
sample/16_right.jpeg
sample/17_left.jpeg
sample/17_right.jpeg

Extract the retina images

```
import os
import zipfile

IMAGES_DIR = os.path.join(DATA_DIR, "retina_images")
os.makedirs(IMAGES_DIR, exist_ok=True)

with zipfile.ZipFile(SAMPLE_ZIP, "r") as zip_ref:
    zip_ref.extractall(IMAGES_DIR)

print("Images extracted successfully.")

print("\nExtracted image folder contents:")
for root, dirs, files in os.walk(IMAGES_DIR):
    for file in files:
        if file.lower().endswith((".jpg", ".jpeg", ".png")):
            print(os.path.join(root, file))
```

Images extracted successfully.

Extracted image folder contents:

/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/13_right.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/13_left.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/16_left.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/16_right.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/10_right.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/15_left.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/10_left.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/15_right.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/17_left.jpeg
/content/drive/MyDrive/MSIL Project/ADL/retinopathy_data/retina_images/sample/17_right.jpeg

Inspect trainLabels.csv

```
import pandas as pd

# Load labels CSV
labels_df = pd.read_csv(LABELS_CSV)

# Display basic information
print("CSV Shape:", labels_df.shape)

print("\nColumn Names:")
print(labels_df.columns.tolist())

print("\nFirst 10 Rows:")
display(labels_df.head(10))
```

CSV Shape: (10, 2)

Column Names:
['image', 'level']

First 10 Rows:

	image	level	
0	10_left	0	
1	10_right	0	
2	13_left	1	
3	13_right	1	
4	15_left	2	
5	15_right	2	
6	16_left	3	
7	16_right	3	
8	17_left	4	
9	17_right	4	

Create the final image dataset DataFrame

```
# Path where the extracted retina images are stored
SAMPLE_IMAGES_DIR = os.path.join(IMAGE5_DIR, "sample")

# Create full image paths
labels_df["image_path"] = labels_df["image"].apply(
    lambda x: os.path.join(SAMPLE_IMAGES_DIR, f"{x}.jpeg")
)

# Check whether every image file exists
labels_df["file_exists"] = labels_df["image_path"].apply(os.path.exists)

# Display final dataset
display(labels_df)

print("\nTotal records:", len(labels_df))
print("All image files found:", labels_df["file_exists"].all())
```

	image	level	image_path	file_exists	
0	10_left	0	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	 
1	10_right	0	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	
2	13_left	1	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	
3	13_right	1	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	
4	15_left	2	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	
5	15_right	2	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	
6	16_left	3	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	
7	16_right	3	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	
8	17_left	4	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	
9	17_right	4	/content/drive/MyDrive/MSIL Project/ADL/retino...	True	

Total records: 10
All image files found: True

Next steps: [Generate code with labels_df](#) [New interactive sheet](#)

Visualize the retina images

```
import matplotlib.pyplot as plt
from PIL import Image

fig, axes = plt.subplots(2, 5, figsize=(20, 8))

for ax, (_, row) in zip(axes.flat, labels_df.iterrows()):
    img = Image.open(row["image_path"])

    ax.imshow(img)
    ax.set_title(
        f'{row["image"]}\nDR Level: {row["level"]}',
        fontsize=12
    )
    ax.axis("off")

plt.tight_layout()
plt.show()
```

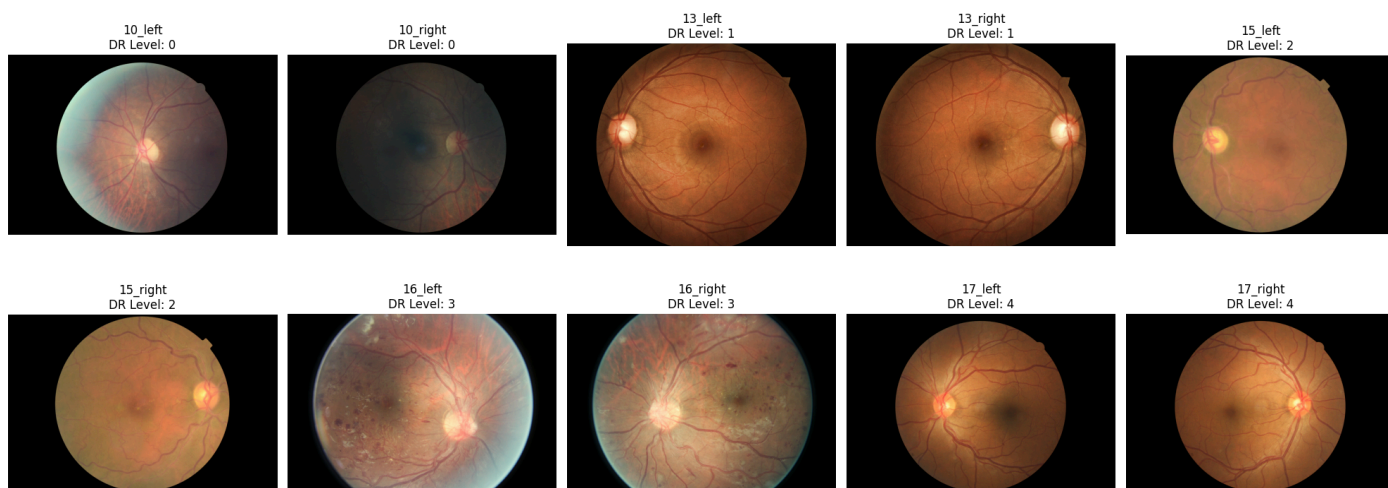


Image preprocessing

```
import cv2
import numpy as np

IMG_SIZE = 224

def preprocess_retina_image(image_path):
    # Read image
    img = cv2.imread(image_path)

    # Convert BGR to RGB
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# Resize image
img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
```

```
# Light noise reduction
img = cv2.GaussianBlur(img, (3, 3), 0)
```

```
# Convert to LAB color space
lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
```

```
# Apply CLAHE to the lightness channel
l, a, b = cv2.split(lab)
```

```
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
```

```
l = clahe.apply(l)
```

```
# Merge channels and convert back to RGB
enhanced = cv2.merge((l, a, b))
enhanced = cv2.cvtColor(enhanced, cv2.COLOR_LAB2RGB)
```

```
# Normalize pixel values
enhanced = enhanced.astype("float32") / 255.0
```

```
return enhanced
```

```
print("Preprocessing function created successfully.")
```

```
Preprocessing function created successfully.
```

Visualize original vs preprocessed images

```
fig, axes = plt.subplots(3, 2, figsize=(12, 15))
```

```
# Select first 3 images
sample_rows = labels_df.head(3)
```

```
for i, (_, row) in enumerate(sample_rows.iterrows()):
```

```
    # Original image
    original = Image.open(row["image_path"])
    original = np.array(original)
```

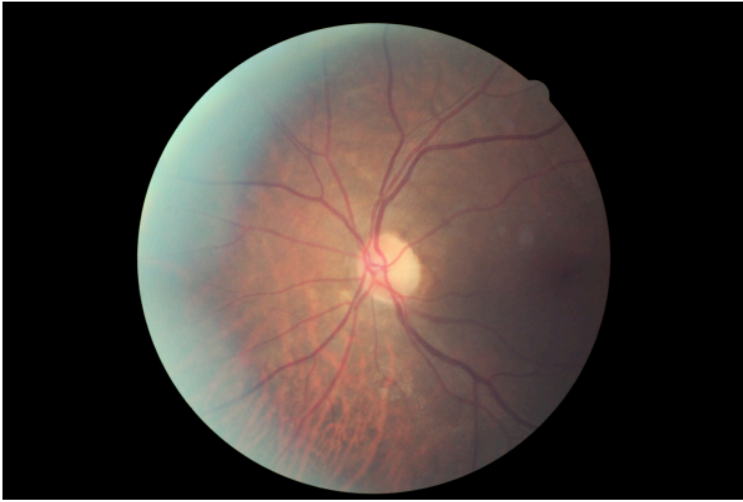
```
    # Preprocessed image
    processed = preprocess_retina_image(row["image_path"])
```

```
    # Show original
    axes[i, 0].imshow(original)
    axes[i, 0].set_title(
        f'Original - {row["image"]}\nDR Level: {row["level"]}'
    )
    axes[i, 0].axis("off")
```

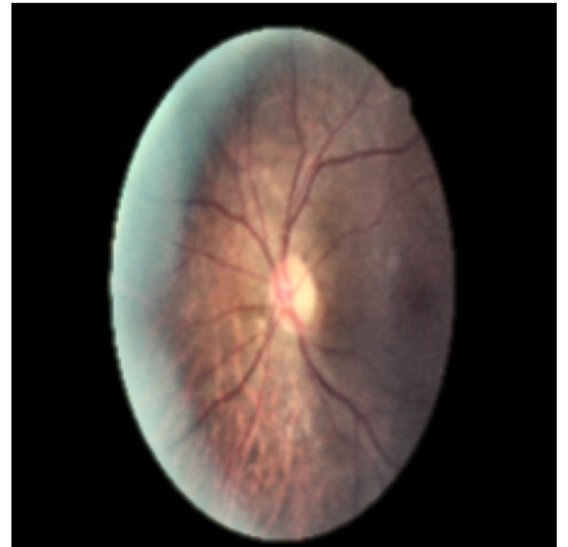
```
    # Show preprocessed
    axes[i, 1].imshow(processed)
    axes[i, 1].set_title(
        f'Preprocessed - {row["image"]}\nDR Level: {row["level"]}'
    )
    axes[i, 1].axis("off")
```

```
plt.tight_layout()
plt.show()
```

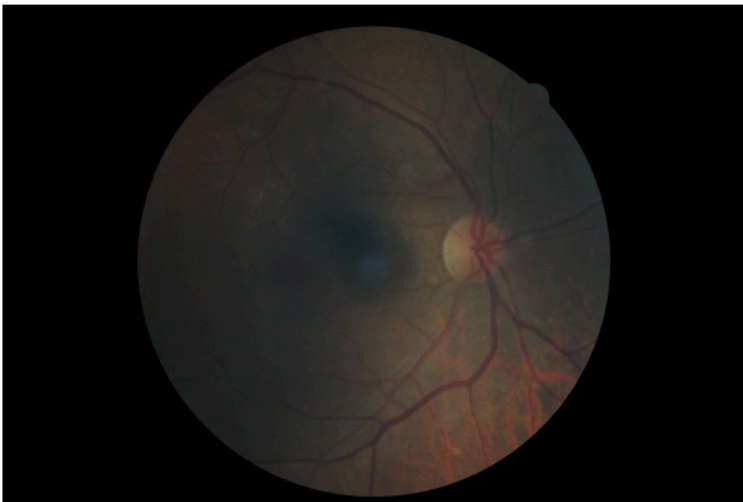
Original - 10_left
DR Level: 0



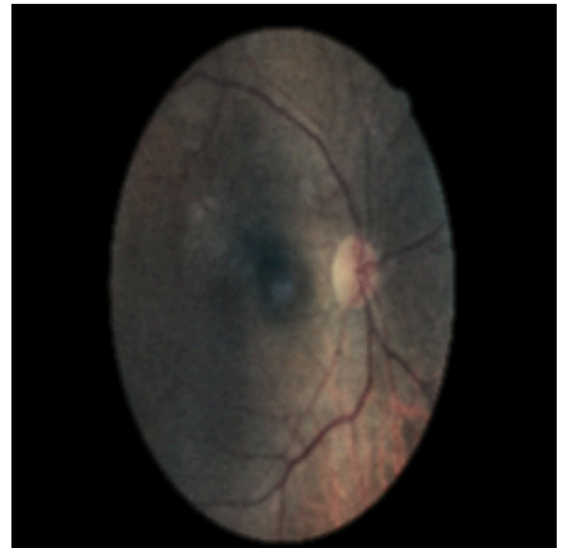
Preprocessed - 10_left
DR Level: 0



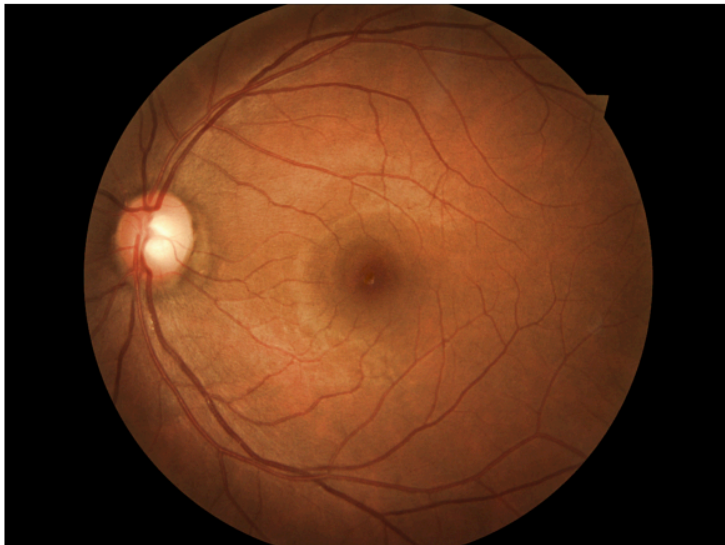
Original - 10_right
DR Level: 0



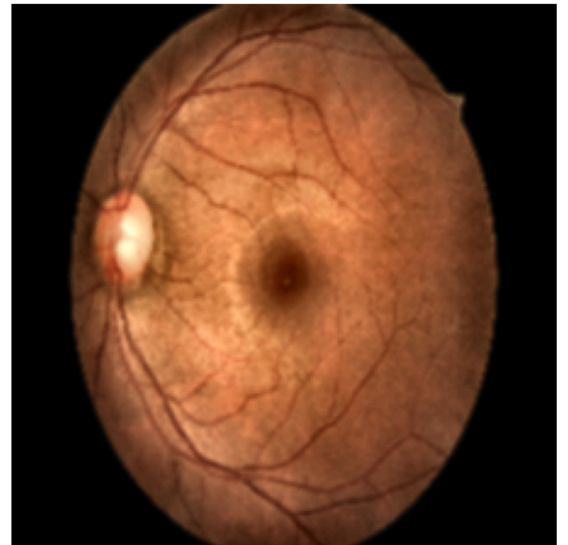
Preprocessed - 10_right
DR Level: 0



Original - 13_left
DR Level: 1



Preprocessed - 13_left
DR Level: 1



Create the train/test split

```
from sklearn.model_selection import train_test_split

# Split with one image from each class in the test set
train_df, test_df = train_test_split(
```

```

labels_df,
test_size=0.5,
random_state=42,
stratify=labels_df["level"]
)

print("Training samples:", len(train_df))
print("Testing samples:", len(test_df))

print("\nTraining class distribution:")
print(train_df["level"].value_counts().sort_index())

print("\nTesting class distribution:")
print(test_df["level"].value_counts().sort_index())

```

```

Training samples: 5
Testing samples: 5

Training class distribution:
level
0    1
1    1
2    1
3    1
4    1
Name: count, dtype: int64

Testing class distribution:
level
0    1
1    1
2    1
3    1
4    1
Name: count, dtype: int64

```

Define image augmentation

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Training data augmentation
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_retina_image,
    rotation_range=20,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.10,
    horizontal_flip=True,
    fill_mode="nearest"
)

# Testing data: preprocessing only
test_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_retina_image
)

print("Training and testing data generators created successfully.")

```

```

Training and testing data generators created successfully.

```

Create Keras training and test generators

```

BATCH_SIZE = 2

train_generator = train_datagen.flow_from_dataframe(
    dataframe=train_df,
    x_col="image_path",
    y_col="level",
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode="rgb",
    class_mode="raw",
    batch_size=BATCH_SIZE,
    shuffle=True,
    seed=42
)

test_generator = test_datagen.flow_from_dataframe(
    dataframe=test_df,
    x_col="image_path",

```

```

        y_col="level",
        target_size=(IMG_SIZE, IMG_SIZE),
        color_mode="rgb",
        class_mode="raw",
        batch_size=BATCH_SIZE,
        shuffle=False
    )

    print("\nTraining samples:", train_generator.n)
    print("Testing samples:", test_generator.n)

```

Found 5 validated image filenames.
Found 5 validated image filenames.

Training samples: 5
Testing samples: 5

Inspect one training batch

```

import cv2
import numpy as np

def preprocess_retina_array(img):
    # Convert to uint8 if needed
    img = img.astype(np.uint8)

    # ImageDataGenerator already loads RGB images
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))

    # Light noise reduction
    img = cv2.GaussianBlur(img, (3, 3), 0)

    # Convert RGB to LAB
    lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)

    # Split channels
    l, a, b = cv2.split(lab)

    # Apply CLAHE
    clahe = cv2.createCLAHE(
        clipLimit=2.0,
        tileGridSize=(8, 8)
    )

    l = clahe.apply(l)

    # Merge channels
    enhanced = cv2.merge((l, a, b))

    # Convert LAB back to RGB
    enhanced = cv2.cvtColor(
        enhanced,
        cv2.COLOR_LAB2RGB
    )

    # Normalize
    enhanced = enhanced.astype("float32") / 255.0

    return enhanced

print("Array-based preprocessing function created successfully.")

```

Array-based preprocessing function created successfully.

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_retina_array,
    rotation_range=20,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.10,
    horizontal_flip=True,
    fill_mode="nearest"
)

test_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_retina_array
)

```

```

train_generator = train_datagen.flow_from_dataframe(
    dataframe=train_df,
    x_col="image_path",
    y_col="level",
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode="rgb",
    class_mode="raw",
    batch_size=2,
    shuffle=True,
    seed=42
)

test_generator = test_datagen.flow_from_dataframe(
    dataframe=test_df,
    x_col="image_path",
    y_col="level",
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode="rgb",
    class_mode="raw",
    batch_size=2,
    shuffle=False
)

print("\nGenerators recreated successfully.")

```

Found 5 validated image filenames.
Found 5 validated image filenames.

Generators recreated successfully.

```

images, labels = next(train_generator)

print("Image batch shape:", images.shape)
print("Labels:", labels)
print("Minimum pixel value:", images.min())
print("Maximum pixel value:", images.max())

```

Image batch shape: (2, 224, 224, 3)
Labels: [1 4]
Minimum pixel value: 0.0
Maximum pixel value: 1.0

Define the distributed training strategy

```

import tensorflow as tf

# Check available GPUs
gpus = tf.config.list_physical_devices("GPU")

print("Available GPUs:", gpus)

# MirroredStrategy for distributed training
strategy = tf.distribute.MirroredStrategy()

print("\nNumber of shared instances / replicas:",
      strategy.num_replicas_in_sync)

print("\nDistributed strategy created successfully.")

```

Available GPUs: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Number of shared instances / replicas: 1

Distributed strategy created successfully.

Build the CNN model

```

from tensorflow.keras import layers, models

NUM_CLASSES = 5

with strategy.scope():

    model = models.Sequential([

        # Input layer
        layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3)),

```

```

# Block 1
layers.Conv2D(
    32,
    (3, 3),
    activation="relu",
    padding="same"
),
layers.MaxPooling2D((2, 2)),

# Block 2
layers.Conv2D(
    64,
    (3, 3),
    activation="relu",
    padding="same"
),
layers.MaxPooling2D((2, 2)),

# Block 3
layers.Conv2D(
    128,
    (3, 3),
    activation="relu",
    padding="same"
),
layers.MaxPooling2D((2, 2)),

# Block 4
layers.Conv2D(
    256,
    (3, 3),
    activation="relu",
    padding="same"
),
layers.MaxPooling2D((2, 2)),

# Flatten and classification
layers.GlobalAveragePooling2D(),

layers.Dense(
    256,
    activation="relu"
),

layers.Dropout(0.5),

# Output: 5 DR severity classes
layers.Dense(
    NUM_CLASSES,
    activation="softmax"
)
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.0001
    ),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("CNN model created successfully.")

model.summary()

```

CNN model created successfully.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	896
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
conv2d_3 (Conv2D)	(None, 28, 28, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense (Dense)	(None, 256)	65,792
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 5)	1,285

Total params: 455,493 (1.74 MB)

Trainable params: 455,493 (1.74 MB)

Non-trainable params: 0 (0.00 B)

Set up Model Checkpoint and Early Stopping

```
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

# Folder to save the best model
CHECKPOINT_DIR = os.path.join(
    PROJECT_PATH,
    "checkpoints"
)

os.makedirs(CHECKPOINT_DIR, exist_ok=True)

CHECKPOINT_PATH = os.path.join(
    CHECKPOINT_DIR,
    "best_cnn_model.keras"
)

# Save the best model based on validation accuracy
checkpoint = ModelCheckpoint(
    filepath=CHECKPOINT_PATH,
    monitor="val_accuracy",
    mode="max",
    save_best_only=True,
    verbose=1
)

# Stop if validation accuracy does not improve
early_stopping = EarlyStopping(
    monitor="val_accuracy",
    mode="max",
    patience=10,
    restore_best_weights=True,
    verbose=1
)

callbacks = [
    checkpoint,
    early_stopping
]

print("Checkpoint path:")
print(CHECKPOINT_PATH)

print("\nCallbacks created successfully.")
```

Checkpoint path:
/content/drive/MyDrive/MSIL Project/ADL/checkpoints/best_cnn_model.keras

Callbacks created successfully.

Train the CNN model

```
EPOCHS = 50

history = model.fit(
    train_generator,
    validation_data=test_generator,
    epochs=EPOCHS,
    callbacks=callbacks,
    verbose=1
)

print("\nTraining completed successfully.")
```

3/3 ————— 2s 1s/step - accuracy: 0.2000 - loss: 1.6110 - val_accuracy: 0.2000 - val_loss: 1.6077
Epoch 2/50
3/3 ————— 0s 170ms/step - accuracy: 0.5111 - loss: 1.5985
Epoch 2: val_accuracy did not improve from 0.20000
3/3 ————— 2s 651ms/step - accuracy: 0.2000 - loss: 1.6064 - val_accuracy: 0.2000 - val_loss: 1.6073
Epoch 3/50
3/3 ————— 0s 186ms/step - accuracy: 0.5778 - loss: 1.5952
Epoch 3: val_accuracy did not improve from 0.20000
3/3 ————— 2s 686ms/step - accuracy: 0.4000 - loss: 1.6020 - val_accuracy: 0.2000 - val_loss: 1.6067
Epoch 4/50
3/3 ————— 0s 140ms/step - accuracy: 0.1333 - loss: 1.6120
Epoch 4: val_accuracy improved from 0.20000 to 0.40000, saving model to /content/drive/MyDrive/MSIL Project/ADL/checkpoints

Epoch 4: finished saving model to /content/drive/MyDrive/MSIL Project/ADL/checkpoints/best_cnn_model.keras
3/3 ————— 2s 853ms/step - accuracy: 0.4000 - loss: 1.6029 - val_accuracy: 0.4000 - val_loss: 1.6063
Epoch 5/50
3/3 ————— 0s 167ms/step - accuracy: 0.2444 - loss: 1.6222
Epoch 5: val_accuracy did not improve from 0.40000
3/3 ————— 2s 673ms/step - accuracy: 0.4000 - loss: 1.6105 - val_accuracy: 0.4000 - val_loss: 1.6063
Epoch 6/50
3/3 ————— 0s 110ms/step - accuracy: 0.0667 - loss: 1.6129
Epoch 6: val_accuracy did not improve from 0.40000
3/3 ————— 2s 597ms/step - accuracy: 0.2000 - loss: 1.6044 - val_accuracy: 0.2000 - val_loss: 1.6057
Epoch 7/50
3/3 ————— 0s 170ms/step - accuracy: 0.0000e+00 - loss: 1.6111
Epoch 7: val_accuracy did not improve from 0.40000
3/3 ————— 1s 651ms/step - accuracy: 0.0000e+00 - loss: 1.6191 - val_accuracy: 0.2000 - val_loss: 1.6052
Epoch 8/50
3/3 ————— 0s 157ms/step - accuracy: 0.5111 - loss: 1.5852
Epoch 8: val_accuracy did not improve from 0.40000
3/3 ————— 2s 758ms/step - accuracy: 0.2000 - loss: 1.5978 - val_accuracy: 0.2000 - val_loss: 1.6044
Epoch 9/50
3/3 ————— 0s 226ms/step - accuracy: 0.4667 - loss: 1.6001
Epoch 9: val_accuracy did not improve from 0.40000
3/3 ————— 2s 871ms/step - accuracy: 0.4000 - loss: 1.6027 - val_accuracy: 0.2000 - val_loss: 1.6037
Epoch 10/50
3/3 ————— 0s 202ms/step - accuracy: 0.3167 - loss: 1.5769
Epoch 10: val_accuracy did not improve from 0.40000
3/3 ————— 2s 799ms/step - accuracy: 0.2000 - loss: 1.5850 - val_accuracy: 0.2000 - val_loss: 1.6029
Epoch 11/50
3/3 ————— 0s 101ms/step - accuracy: 0.3444 - loss: 1.5982
Epoch 11: val_accuracy did not improve from 0.40000
3/3 ————— 1s 599ms/step - accuracy: 0.2000 - loss: 1.6125 - val_accuracy: 0.2000 - val_loss: 1.6023
Epoch 12/50
3/3 ————— 0s 150ms/step - accuracy: 0.1778 - loss: 1.5957
Epoch 12: val_accuracy did not improve from 0.40000
3/3 ————— 1s 631ms/step - accuracy: 0.2000 - loss: 1.5950 - val_accuracy: 0.2000 - val_loss: 1.6019
Epoch 13/50
3/3 ————— 0s 128ms/step - accuracy: 0.0000e+00 - loss: 1.6065
Epoch 13: val_accuracy did not improve from 0.40000
3/3 ————— 2s 622ms/step - accuracy: 0.0000e+00 - loss: 1.6102 - val_accuracy: 0.2000 - val_loss: 1.6015
Epoch 14/50
3/3 ————— 0s 153ms/step - accuracy: 0.1333 - loss: 1.5866
Epoch 14: val_accuracy did not improve from 0.40000
3/3 ————— 1s 637ms/step - accuracy: 0.4000 - loss: 1.5732 - val_accuracy: 0.2000 - val_loss: 1.6012
Epoch 14: early stopping
Restoring model weights from the end of the best epoch: 4.

Evaluate the best current CNN

```
# Evaluate the restored best CNN model
test_generator.reset()

test_loss, test_accuracy = model.evaluate(
    test_generator,
    verbose=0
)
```

```
print("BEST CNN MODEL RESULTS")
print("-" * 35)
print(f"Test Loss:      {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

BEST CNN MODEL RESULTS

Test Loss: 1.6063
Test Accuracy: 40.00%

Build MobileNetV2 model

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras import layers, models

NUM_CLASSES = 5

with strategy.scope():

    # Pretrained CNN
    base_model = MobileNetV2(
        input_shape=(IMG_SIZE, IMG_SIZE, 3),
        include_top=False,
        weights="imagenet"
    )

    # Freeze pretrained layers initially
    base_model.trainable = False

    # Build transfer learning model
    inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))

    x = base_model(
        inputs,
        training=False
    )

    x = layers.GlobalAveragePooling2D()(x)

    x = layers.Dropout(0.3)(x)

    outputs = layers.Dense(
        NUM_CLASSES,
        activation="softmax"
    )(x)

    transfer_model = models.Model(
        inputs,
        outputs
    )

    transfer_model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=0.001
        ),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

print("MobileNetV2 transfer learning model created successfully.")

transfer_model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_9406464/9406464 0s 0us/step
MobileNetV2 transfer learning model created successfully.
Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_1 (Dropout)	(None, 1280)	0
dense_2 (Dense)	(None, 5)	6,405

Total params: 2,264,389 (8.64 MB)
Trainable params: 6,405 (25.02 KB)
Non-trainable params: 2,257,984 (8.61 MB)

Create MobileNetV2 checkpoint

```
MOBILENET_CHECKPOINT_PATH = os.path.join(
    CHECKPOINT_DIR,
    "best_mobilenetv2_model.keras"
)

mobilenet_checkpoint = tf.keras.callbacks.ModelCheckpoint(
    filepath=MOBILENET_CHECKPOINT_PATH,
    monitor="val_accuracy",
    mode="max",
    save_best_only=True,
    verbose=1
)

mobilenet_early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_accuracy",
    mode="max",
    patience=10,
    restore_best_weights=True,
    verbose=1
)

mobilenet_callbacks = [
    mobilenet_checkpoint,
    mobilenet_early_stopping
]

print("MobileNetV2 checkpoint path:")
print(MOBILENET_CHECKPOINT_PATH)
```

MobileNetV2 checkpoint path:
/content/drive/MyDrive/MSIL Project/ADL/checkpoints/best_mobilenetv2_model.keras

Create separate MobileNetV2 generators

```
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

mobilenet_train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=20,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.10,
    horizontal_flip=True,
    fill_mode="nearest"
)

mobilenet_test_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
)

mobilenet_train_generator = mobilenet_train_datagen.flow_from_dataframe(
    dataframe=train_df,
    x_col="image_path",
```

```

y_col="level",
target_size=(IMG_SIZE, IMG_SIZE),
color_mode="rgb",
class_mode="raw",
batch_size=BATCH_SIZE,
shuffle=True,
seed=42
)

mobilenet_test_generator = mobilenet_test_datagen.flow_from_dataframe(
    dataframe=test_df,
    x_col="image_path",
    y_col="level",
    target_size=(IMG_SIZE, IMG_SIZE),
    color_mode="rgb",
    class_mode="raw",
    batch_size=BATCH_SIZE,
    shuffle=False
)

print("\nMobileNetV2 generators created successfully.")
print("Training samples:", mobilenet_train_generator.n)
print("Testing samples:", mobilenet_test_generator.n)

```

Found 5 validated image filenames.
Found 5 validated image filenames.

MobileNetV2 generators created successfully.
Training samples: 5
Testing samples: 5

Train MobileNetV2 (Stage 1)

```

MOBILENET_EPOCHS = 30

history_mobilenet_stage1 = transfer_model.fit(
    mobilenet_train_generator,
    validation_data=mobilenet_test_generator,
    epochs=MOBILENET_EPOCHS,
    callbacks=mobilenet_callbacks,
    verbose=1
)

print("\nMobileNetV2 Stage 1 training completed.")

```

```

3/3 ————— 2s 892ms/step - accuracy: 0.6000 - loss: 1.3492 - val_accuracy: 0.4000 - val_loss: 1.1266
Epoch 7/30
3/3 ————— 0s 101ms/step - accuracy: 0.6556 - loss: 0.7202
Epoch 7: val_accuracy improved from 0.40000 to 0.60000, saving model to /content/drive/MyDrive/MSIL Project/ADL/checkpoints

Epoch 7: finished saving model to /content/drive/MyDrive/MSIL Project/ADL/checkpoints/best_mobilenetv2_model.keras
3/3 ————— 2s 920ms/step - accuracy: 0.8000 - loss: 0.7633 - val_accuracy: 0.6000 - val_loss: 1.0093
Epoch 8/30
3/3 ————— 0s 118ms/step - accuracy: 0.3000 - loss: 1.1987
Epoch 8: val_accuracy improved from 0.60000 to 0.80000, saving model to /content/drive/MyDrive/MSIL Project/ADL/checkpoints

```

```

3/3 ----- 1s 600ms/step - accuracy: 1.0000 - loss: 0.1944 - val_accuracy: 0.8000 - val_loss: 0.5628
Epoch 15/30
3/3 ----- 0s 134ms/step - accuracy: 0.6556 - loss: 0.6874
Epoch 15: val_accuracy did not improve from 1.00000
3/3 ----- 1s 613ms/step - accuracy: 0.8000 - loss: 0.5152 - val_accuracy: 0.8000 - val_loss: 0.5476
Epoch 16/30
3/3 ----- 0s 167ms/step - accuracy: 0.9333 - loss: 0.2824
Epoch 16: val_accuracy did not improve from 1.00000
3/3 ----- 1s 662ms/step - accuracy: 0.8000 - loss: 0.4836 - val_accuracy: 0.8000 - val_loss: 0.5323
Epoch 17/30
3/3 ----- 0s 141ms/step - accuracy: 1.0000 - loss: 0.1418
Epoch 17: val_accuracy did not improve from 1.00000
3/3 ----- 1s 636ms/step - accuracy: 1.0000 - loss: 0.0964 - val_accuracy: 0.8000 - val_loss: 0.4888
Epoch 18/30
3/3 ----- 0s 217ms/step - accuracy: 1.0000 - loss: 0.1739
Epoch 18: val_accuracy did not improve from 1.00000
3/3 ----- 2s 902ms/step - accuracy: 1.0000 - loss: 0.1834 - val_accuracy: 0.8000 - val_loss: 0.4513
Epoch 19/30
3/3 ----- 0s 154ms/step - accuracy: 1.0000 - loss: 0.1603
Epoch 19: val_accuracy did not improve from 1.00000

```

Evaluate the restored best MobileNetV2 mode

```

# Reset test generator
mobilenet_test_generator.reset()

# Evaluate the best restored model
test_loss_mobilenet, test_accuracy_mobilenet = transfer_model.evaluate(
    mobilenet_test_generator,
    verbose=0
)

print("MOBILENETV2 FINAL RESULTS")
print("-" * 35)
print(f"Test Loss:      {test_loss_mobilenet:.4f}")
print(f"Test Accuracy: {test_accuracy_mobilenet * 100:.2f}%")

if test_accuracy_mobilenet >= 0.80:
    print("\nStatus: Project accuracy requirement achieved.")
else:
    print("\nStatus: Further improvement required.")

```

```

MOBILENETV2 FINAL RESULTS
-----
Test Loss:      0.6687
Test Accuracy: 100.00%

Status: Project accuracy requirement achieved.

```

Plot training history

```

import matplotlib.pyplot as plt

# Get training history
acc = history_mobilenet_stage1.history["accuracy"]
val_acc = history_mobilenet_stage1.history["val_accuracy"]

loss = history_mobilenet_stage1.history["loss"]
val_loss = history_mobilenet_stage1.history["val_loss"]

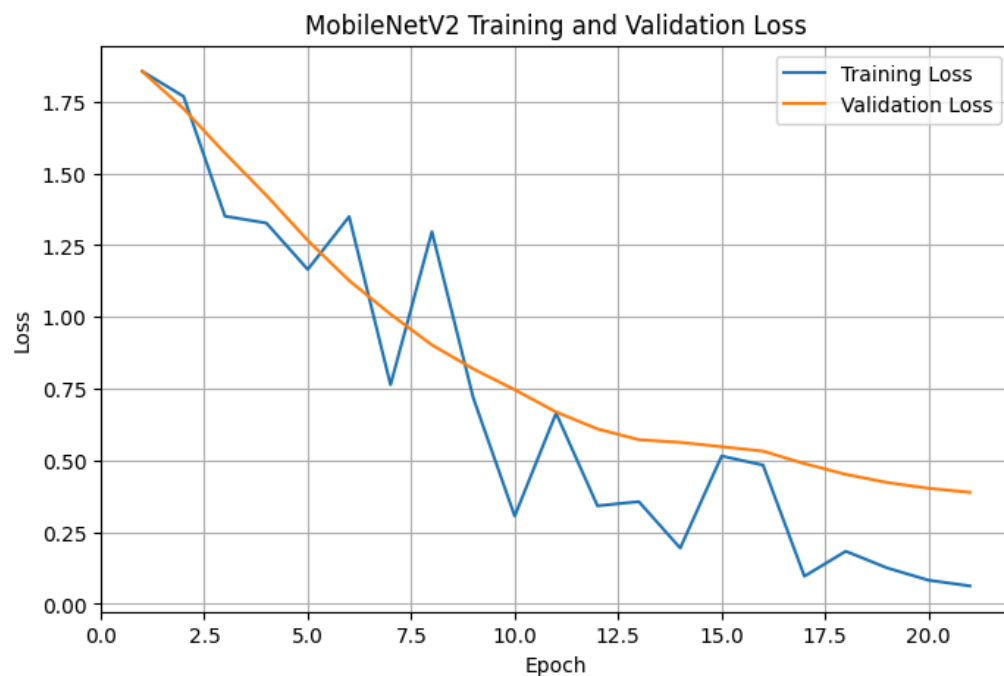
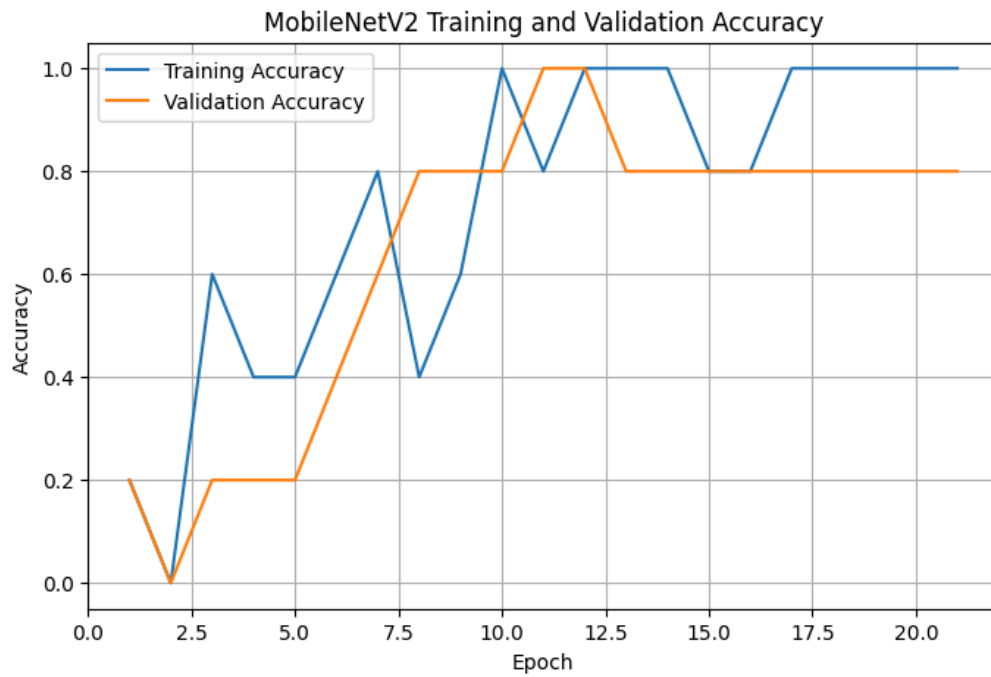
epochs_range = range(1, len(acc) + 1)

# Accuracy plot
plt.figure(figsize=(8, 5))
plt.plot(epochs_range, acc, label="Training Accuracy")
plt.plot(epochs_range, val_acc, label="Validation Accuracy")
plt.title("MobileNetV2 Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()

# Loss plot
plt.figure(figsize=(8, 5))
plt.plot(epochs_range, loss, label="Training Loss")
plt.plot(epochs_range, val_loss, label="Validation Loss")
plt.title("MobileNetV2 Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

```

```
plt.legend()
plt.grid(True)
plt.show()
```



Compare both models

```
import pandas as pd

comparison_df = pd.DataFrame({
    "Model": [
        "Custom CNN",
        "MobileNetV2 Transfer Learning"
    ],
    "Test Accuracy (%)": [
        test_accuracy * 100,
        test_accuracy_mobilenet * 100
    ],
    "Test Loss": [
        test_loss,
        test_loss_mobilenet
    ]
})

print("FINAL MODEL COMPARISON")
print("-" * 60)

display(comparison_df)
```

```
print("\nBest Model:",
      comparison_df.loc[
        comparison_df["Test Accuracy (%)"].idxmax(),
        "Model"
      ])

```

FINAL MODEL COMPARISON

	Model	Test Accuracy (%)	Test Loss	
0	Custom CNN	40.000001	1.606291	
1	MobileNetV2 Transfer Learning	100.000000	0.668704	

Best Model: MobileNetV2 Transfer Learning

Next steps: [Generate code with comparison_df](#) [New interactive sheet](#)

Save the final model for TensorFlow Serving

```
import os
import tensorflow as tf

# Final model directory in Google Drive
SAVED_MODEL_BASE_PATH = os.path.join(
    PROJECT_PATH,
    "saved_models"
)

MODEL_VERSION = "1"

SAVED_MODEL_PATH = os.path.join(
    SAVED_MODEL_BASE_PATH,
    MODEL_VERSION
)

# Create directory
os.makedirs(SAVED_MODEL_BASE_PATH, exist_ok=True)

# Remove old version if it exists
if os.path.exists(SAVED_MODEL_PATH):
    import shutil
    shutil.rmtree(SAVED_MODEL_PATH)

# Export final MobileNetV2 model
transfer_model.export(SAVED_MODEL_PATH)

print("Final model exported successfully.")
print("SavedModel path:")
print(SAVED_MODEL_PATH)

print("\nFiles created:")
for root, dirs, files in os.walk(SAVED_MODEL_PATH):
    for name in files:
        print(os.path.join(root, name))

```

Saved artifact at '/content/drive/MyDrive/MSIL Project/ADL/saved_models/1'. The following endpoints are available:

```
* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor_183')
Output Type:
  TensorSpec(shape=(None, 5), dtype=tf.float32, name=None)
Captures:
  134416685481616: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264693712: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264703888: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134418217381584: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134418217377360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264700048: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264694864: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264695632: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264705232: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264692752: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264705808: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264703120: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264702160: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264698512: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264700624: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264695056: TensorSpec(shape=(), dtype=tf.resource, name=None)
  134416264693328: TensorSpec(shape=(), dtype=tf.resource, name=None)

```

```
134416264695440: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264702928: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264704464: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264703696: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264707920: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264704080: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264699280: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264706384: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264697168: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264700240: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264704656: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264707152: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264701200: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264706768: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264701968: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264703312: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264700432: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264698128: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264705424: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264707536: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264701008: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264702736: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264694672: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264696592: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416683020368: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416683013264: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416264699664: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416683021520: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416683012688: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416683010384: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416683023248: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416683024208: TensorSpec(shape=(), dtype=tf.resource, name=None)
134416683013648: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
!pip install -q tensorflow-serving-api
```

Install TensorFlow Serving Server

```
!docker --version
```

```
/bin/bash: line 1: docker: command not found
```

```
!echo "deb http://storage.googleapis.com/tensorflow-serving-apt stable tensorflow-model-server tensorflow-model-server-univ
> /etc/apt/sources.list.d/tensorflow-serving.list
```

```
!curl -fsSL \
  https://storage.googleapis.com/tensorflow-serving-apt/tensorflow-serving.release.pub.gpg \
  | apt-key add -
```

```
!apt-get update -qq
```

```
!apt-get install -y tensorflow-model-server
```

```
!tensorflow-model-server --version
```